九阶 · Linux 与 Python 自学资料

09

K8s 入门

从 Pod 到可观察、可恢复的部署

教科书

AYMLiu-Pzw · 2026-10-21

九阶 · Linux 与 Python 自学资料

第 09 册

K8s 入门

从 Pod 到可观察、可恢复的部署

资料类型	教科书
适用读者	完成监控阶段 5 的学习者
发布日期	2026-10-21
作者署名	AYMLiu-Pzw

阅读导航

按章级标题定位正文；PDF 阅读器中的书签同时提供章、节两级入口。

1. 6-01 打开 Killercoda，认识 kubectl
2. 6-02 部署第一个 Pod（Deployment + Service）
3. 6-03 看：get / describe / logs
4. 6-04 进容器：exec
5. 6-05 自愈：删掉 Pod 看它自动重建
6. 6-06 故意弄坏：镜像名写错，排查 ImagePullBackOff
7. 6-07 资源配额与调度：requests / limits
8. 6-08 阶段 6 · 学习总结练习：部署 → 弄坏 → 排查 → 留一份踩坑记录

09 • K8s 入门

本册是阶段 6 的正文，从 6-01 起步；配套与路线见《17-关卡总表》。
本册共 8 关（6-01 ~ 6-08）。目标：能在 Killercoda 沙箱里部署 Pod / Service，会看“三只眼”（get / describe / logs），能排查 ImagePullBackOff，懂 requests/limits 与调度。
用浏览器沙箱练，不用在本机装 minikube。
本册是什么：K8s —— 把容器编成集群：滚动升级、自愈、资源配额。
动手做实验时照这两条来：实验闭环（先记录原状态 → 操作 → 按输出 / 退出码验证 → 恢复现场）和踩坑记录位（现象 → 原因 → 解决 → 一句话总结）——不必逐关都写，做完整册回头总结一次就够。

6-01 打开 Killercoda，认识 kubectl

先说环境：为什么用 Killercoda：K8s 最少要几台机器。在自己电脑的虚拟机里装 minikube 经常卡在嵌套虚拟化上，一卡就是半天。**Killercoda 是浏览器里的免费沙箱 —— 打开就有集群，零安装、零资源占用，关掉就没了。**

步骤 1：进沙箱

```
浏览器打开 https://killercoda.com/playgrounds → 选 "Kubernetes"（不是 Docker 那个）
等 10 秒左右，右侧出现终端：
$ kubectl version
Client Version: v1.37.x
Server Version: v1.37.x
$ kubectl get nodes
NAME           STATUS   ROLES           AGE   VERSION
controlplane   Ready    control-plane   1m    v1.37.1
node01         Ready    <none>          30s   v1.37.1
```

（节点名、数量和版本都随沙箱变化。示例写明 2026-10-21 的受支持主线快照为 1.37.x；**以你实际运行 `kubectl version` / `kubectl get nodes` 的结果为准**，并把实际版本记录到学习日志；只要节点 STATUS 是 Ready 就能继续本册。）

解读：**STATUS 是 Ready** 就说明节点已就绪。`kubectl` 是主要命令行客户端，后面操作主要靠它；Kubernetes API、client-go、控制器和 GitOps 工具也都能与集群交互。**注意 Roles 那列写着 control-plane** —— 这是管理面角色，node01 是工作节点，这是集群的基础分工。

步骤 2：认识三个名词（先记住这三个，别的慢慢来）

名词	是什么	一句话类比
Pod	K8s 里最小的运行单位（一个或多个容器）	一个"工人"
Deployment	"我要几个副本、用什么镜像"的说明书	工头下的"招 2 个人"的指令
Service	一组 Pod 的稳定入口（Pod 会变，入口不变）	公司的"前台电话"

解读：Pod 随时可能被重建（IP 会变），**应用间通信应优先使用 Service 提供的稳定入口**；这不等于永远不能访问 Pod——临时调试、进程诊断和 sidecar 场景都可能直接针对 Pod。Service 是稳定入口，Pod 是实际运行实例，两者分工不同。下一关就动手。

步骤 3：看看空集群里有什么

```
$ kubectl get pods -A
NAMESPACE     NAME                                   READY   STATUS    RESTARTS
kube-system   coredns-6f6b679f8f-abcde               1/1     Running   0
kube-system   kube-proxy-xyz12                       1/1     Running   0
...

$ kubectl get ns
NAME              STATUS   AGE
default           Active   3m
kube-system       Active   3m
```

解读：**namespace（命名空间）**是集群里的"分区"，系统自己的东西住在 `kube-system`，你的东西默认放 `default`。`-A` 表示"看所有分区"。`READY 1/1` = 一个容器里该起的都起来了。

完成判定

· 能连上沙箱并跑通 `kubectl version` 与 `kubectl get nodes`，记录实际版本 · 能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇） · 能说出 Pod / Deployment / Service 各解决什么问题 · 能说出应用流量为什么应优先走 Service，以及什么时候需要直接访问 Pod · 能说出 namespace 是干什么的
回滚方式：沙箱是临时的，关掉浏览器就全清；不占本机资源，无回滚负担。

6-02 部署第一个 Pod（Deployment + Service）

步骤 1：写 Deployment 文件

```
$ cat > web.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2                # 我要 2 个副本
  selector:
    matchLabels:
      app: web
  template:                  # 下面是"每个副本长什么样"
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
          ports:
            - containerPort: 80
EOF
```

解读：这份 YAML 的骨架和 compose 一样（缩进即层级）。三处关键：`replicas: 2`（要几个副本）、`selector.matchLabels` 与 `template.metadata.labels` **必须对上**（Deployment 靠标签找自己的 Pod）、`containers` 下面是"用哪个镜像、开哪个端口"。

步骤 2：apply 上去

```
$ kubectl apply -f web.yaml
deployment.apps/web created

$ kubectl get deploy,pods
NAME                  READY   UP-TO-DATE   AVAILABLE
deployment.apps/web   2/2     2            2

NAME                       READY   STATUS    RESTARTS   AGE
pod/web-6d8f7c9b4d-7x2kl   1/1     Running   0          15s
pod/web-6d8f7c9b4d-q9m4t   1/1     Running   0          15s
```

解读：**2/2 就是"要 2 个、好了 2 个"**。Pod 名字里那串随机字符是 K8s 自动生成的（每次重建都不一样 —— 所以别记 Pod 名）。

步骤 3：给它一个稳定入口（Service）

```
$ kubectl expose deployment web --name=web-svc --port=80 --target-port=80
service/web-svc exposed

$ kubectl get svc
NAME      TYPE        CLUSTER-IP      PORT(S)   AGE
web-svc   ClusterIP   10.96.142.11    80/TCP    5s

$ kubectl run tmp --rm -it --image=busybox --restart=Never -- wget -qO- http://web-svc | head -3   # --rm 用完自动删；--restart=Never 只跑一次、不重建
<!DOCTYPE html>
<html>
<title>Welcome to nginx!</title>
```

解读：expose 一条命令就建了个 Service（刚才那张"前台电话"）。**集群内的其他 Pod 用 `web-svc` 这个名字就能访问到那 2 个 Pod**（和 4-08 里 compose 的服务名一个道理）。最后那条 `wget` 是"从集群内部访问一次"，看到 nginx 首页就说明通了。

步骤 4：看它到底起了什么

```
$ kubectl get all
NAME                          READY   STATUS    RESTARTS
pod/web-6d8f7c9b4d-7x2kl       1/1     Running   0
pod/web-6d8f7c9b4d-q9m4t       1/1     Running   0

NAME                 TYPE        CLUSTER-IP     PORT(S)
service/web-svc      ClusterIP   10.96.142.11   80/TCP
service/kubernetes   ClusterIP   10.96.0.1      443/TCP

NAME                  READY   UP-TO-DATE   AVAILABLE
deployment.apps/web   2/2     2            2
```

解读：`get all` 一次把 Pod、Service、Deployment 都列出来 —— 记住这一条，排障第一步就够用了。

完成判定

· 能不看教材写出一个 2 副本的 Deployment · 能说出 `selector` 和 `labels` 为什么必须对上 · 能用 Service 名字从集群内访问到服务 · 能用 `kubectl get all` 说清现在集群里有什么

回滚方式：`kubectl delete -f web.yaml` 和 `kubectl delete svc web-svc`；沙箱关掉即清空。

6-03 看：get / describe / logs

这一关学的是"三只眼睛"：集群出问题时，你靠这三条命令把盲盒打开。

眼睛 ①：get —— 现在是什么状态

```
$ kubectl get pods -o wide
NAME                   READY   STATUS    RESTARTS   AGE   IP           NODE
web-6d8f7c9b4d-7x2kl   1/1     Running   0          3m    10.244.1.5   node01
web-6d8f7c9b4d-q9m4t   1/1     Running   0          3m    10.244.0.4   controlplane

$ kubectl get pods -w          # 盯着看变化，Ctrl+C 退出
web-6d8f7c9b4d-7x2kl   1/1     Running   0     3m
web-6d8f7c9b4d-q9m4t   0/1     Terminating   0   4m
web-6d8f7c9b4d-new1a   0/1     ContainerCreating
```

解读：`-o wide` 多给你两列：**Pod 的 IP 和它在哪台节点上**（调度结果就看这里）。`-w` 是"实时刷新"，删 Pod 时你会看到 Terminating → 新 Pod ContainerCreating 的过程（下一关的自愈就是这个）。

眼睛 ②：describe —— 为什么是这个状态

```
$ kubectl describe pod web-6d8f7c9b4d-7x2kl | tail -20
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  4m    default-scheduler  Successfully assigned default/web-... to node01
  Normal  Pulling    4m    kubelet            Pulling image "nginx:alpine"
  Normal  Pulled     4m    kubelet            Successfully pulled image
  Normal  Created    4m    kubelet            Created container nginx
  Normal  Started    4m    kubelet            Started container nginx
```

解读：**describe 里的 Events 通常要优先看** —— 它是"时间线"：什么时候被调度、什么时候拉镜像、什么时候起来，很多故障原因会出现在这里。但它不是唯一重点：顶部的 Conditions、容器状态、重启次数，以及 `logs --previous` 同样可能给出根因（第 6 关的 ImagePullBackOff 主要靠 Events 定位）。

眼睛 ③：logs —— 容器里那个程序说了什么

```
$ kubectl logs web-6d8f7c9b4d-7x2kl | tail -3
2026/10/09 22:10:01 [notice] 1#1: start worker process 29

$ kubectl logs -f deploy/web          # 按 Deployment 看（它会挑一个 Pod），-f 实时跟
$ kubectl logs web-6d8f7c9b4d-7x2kl --previous   # 容器重启过？看"上一次"的日志
```

解读：三个细节要记住 —— ① 可以直接写 `deploy/web` 不用抄 Pod 名；② `-f` 实时跟（和 `tail -f` 一样）；③ **`--previous` 是排"容器反复重启"的关键**：当前这次如果刚起来，日志是空的，原因在上一次里。

三只眼睛合起来用

```
Pod 不对劲
 ① kubectl get pods -o wide      → 什么状态？在哪台机器？（Pending / Running / CrashLoopBackOff…）
 ② kubectl describe pod X | tail → Events 里为什么？（调度失败 / 拉镜像失败 / 起不来）
 ③ kubectl logs X [--previous]   → 程序自己报了什么错？
```

完成判定

· 能用 -o wide 说出 Pod 被调度到哪台节点 · 能用 describe 的 Events 讲出一个 Pod 的启动时间线 · 能用 logs 看 Deployment 的日志 · 能说出"容器反复重启"时该加哪个参数（--previous）

回滚方式：只读操作，无风险。

6-04 进容器：exec

步骤 1：进到 Pod 里面看一眼

```
$ kubectl exec -it web-6d8f7c9b4d-7x2kl -- sh
/ # hostname
web-6d8f7c9b4d-7x2kl
/ # ls /etc/nginx
conf.d          mime.types      nginx.conf
/ # cat /etc/nginx/conf.d/default.conf | head -5
server {
    listen       80;
    server_name  localhost;
/ # exit
```

解读：kubectl exec -it Pod名 -- 命令，**注意那个 -- 不能少**：它后面才是"在容器里执行什么"。进容器看现场是排障的常用手段（和 docker exec 一个思路）。

一个容易懵的报错：exec 一只**正在重启**的 Pod（CrashLoopBackOff / Restarting）会报 **container not found ("nginx")**——不是命令写错，是容器此刻不在。先 kubectl get pods 看状态：回到 Running 再进；一直起不来就先按 6-06 的排查链修。

步骤 2：不进容器也能干活

```
$ kubectl exec web-6d8f7c9b4d-7x2kl -- nginx -v
nginx version: nginx/1.27.2

$ kubectl exec deploy/web -- ls /usr/share/nginx/html
50x.html
index.html
```

解读：**不加 -it 就是"执行一条命令就出来"** —— 写排查脚本时常用这种形式。deploy/web 也能用（K8s 帮你挑一个 Pod）。

步骤 3：改点东西，看看会发生什么（重要概念）

```
$ POD=$(kubectl get pod -l app=web -o jsonpath='{.items[0].metadata.name}')
$ kubectl exec "$POD" -- sh -c 'echo hello > /usr/share/nginx/html/test.html'
$ kubectl exec "$POD" -- ls /usr/share/nginx/html
50x.html  index.html  test.html

$ OLD_PODS=$(kubectl get pods -l app=web -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}
{end}')
$ kubectl delete pod "$POD"                    # 只删刚才写文件的那个 Pod
$ NEW_POD=""
$ for i in $(seq 1 60); do
>   while IFS= read -r candidate; do
>     [ -z "$candidate" ] && continue
>     if ! printf '%s\n' "$OLD_PODS" | grep -Fxq "$candidate" && \
>        [ "$(kubectl get pod "$candidate" -o jsonpath='{.status.conditions[?
(@.type=="Ready")].status}')" = "True" ]; then
>       NEW_POD="$candidate"; break
>     fi
>   done < <(kubectl get pods -l app=web -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}
{end}')
>   [ -n "$NEW_POD" ] && break
>   sleep 2
> done
$ if [ -z "$NEW_POD" ]; then echo "120 秒内没有等到不在旧名单中的 Ready Pod"; exit 1; fi
$ printf '替代 Pod：%s\n' "$NEW_POD"
$ kubectl exec "$NEW_POD" -- ls /usr/share/nginx/html
50x.html  index.html                           ← test.html 没了
```

解读：先保存确定的 POD 名和删除前的 Pod 名单。删除后循环检查：必须出现一个**不在旧名单中且 Ready** 的新 Pod，才把它保存为 NEW_POD。这样检查和写入不会落到旧 Pod；只按创建时间取第一条仍可能遇到旧 Pod。**容器里的改动"长"在 Pod 上，Pod 一重建就没了** —— 和 4-03 学的"容器数据会消失"是同一件事，只是换到了 K8s 里。要持久化，就得用 Volume（和 Docker 的数据卷对应）。**这条实战里常踩。**

完成判定

· 能进 Pod 看文件并安全退出 · 能说出 -- 的作用 · 能不进容器直接执行一条命令 · 能说清"为什么在容器里改的东西 Pod 一重建就没了"，以及该用什么解决

回滚方式：删掉的 Pod 会自动重建（这就是下一关的内容）；文件改动随 Pod 一起消失，无残留。

6-05 自愈：删掉 Pod 看它自动重建

这一关是"集群自愈能力"的最小现场：你什么都不用做，删掉一个 Pod，看它自己长回来。

步骤 1：先数清楚

```
$ kubectl get pods
NAME                   READY   STATUS    AGE
web-6d8f7c9b4d-7x2kl   1/1     Running   5m
web-6d8f7c9b4d-q9m4t   1/1     Running   5m

$ kubectl get deploy web -o jsonpath='{.status.readyReplicas}{"\n"}'   # jsonpath：按路径从返回的 JSON 里取值（.status.readyReplicas = 就绪副本数）
2
```

步骤 2：删掉一个，盯着看

```
$ kubectl delete pod web-6d8f7c9b4d-7x2kl
pod "web-6d8f7c9b4d-7x2kl" deleted

$ kubectl get pods -w
web-6d8f7c9b4d-q9m4t   1/1     Running             5m
web-6d8f7c9b4d-pp8mk   0/1     ContainerCreating   2s
web-6d8f7c9b4d-pp8mk   1/1     Running             6s      ← 示例为 6 秒；实际以控制器和集群为准
^C
```

解读：**Deployment 说"我要 2 个副本"，一旦发现只剩 1 个，就立刻建一个新的。**这就是"控制器"在干活：**声明式 —— 你只说要什么状态，K8s 负责把现实拉回那个状态**（不是叫你去启一个）。这一点是 K8s 和"手敲 docker run"最大的区别。

步骤 3：换个视角看它想要的状态

```
$ kubectl get deploy web -o yaml | grep -A3 status
status:
  availableReplicas: 2
  readyReplicas: 2
  replicas: 2            ← 现状 = 期望（spec 里的 2）
```

解读：spec 是"我想要什么"，status 是"现在是什么"。**K8s 的全部工作就是让 status 追上 spec** —— 记住这一句，后面所有资源都按这个套路理解。

完成判定

· 删掉一个 Pod 并看到它被自动重建（截图/记录） · 能说出"声明式"和"命令式"的区别（用删 Pod 这件事举例子） · 能说出 spec 与 status 各是什么 · 能说出这一步在生产里有什么用

回滚方式：不用回滚 —— 实验本身就是删除，集群自己恢复。

6-06 故意弄坏：镜像名写错，排查 ImagePullBackOff

这一关是整册的重点：亲手制造一个真实故障，再靠三条眼（get / describe / logs）把它查出来 —— 这就是“面对没见过的故障看推理路径”的标准练法。

步骤 1：把镜像名写错一个字母

前置检查：本关依赖 6-02 留在当前沙箱里的 `web.yaml`。重开沙箱、隔天回来或从目录直接进入本关时，先重建文件；不要假设旧现场还在。

```
$ test -f web.yaml && echo "OK：当前沙箱仍有 web.yaml"
$ test -f web.yaml || echo "缺少 web.yaml：请执行下面的重建块"

# 仅在上一行提示缺少文件时，执行这个重建块
$ cat > web.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
          ports:
            - containerPort: 80
EOF
$ kubectl apply -f web.yaml
$ sed -i 's/nginx:alpine/nginx:alpne/' web.yaml      # alpne 是错的
$ kubectl apply -f web.yaml
deployment.apps/web configured

$ kubectl get pods
NAME                   READY   STATUS             RESTARTS   AGE
web-7c9d4b6f8a-2k9xz   0/1     ImagePullBackOff   0          20s
web-6d8f7c9b4d-q9m4t   1/1     Running            0          12m
web-6d8f7c9b4d-x7v2p   1/1     Running            0          12m
```

解读：**ImagePullBackOff** 直译"拉镜像退避" —— K8s 试了几次拉不到那个镜像，就先歇一会儿再试（退避）。`replicas: 2` 指期望就绪副本数，不是“任何时刻 Pod 总数必须恰好为 2”：滚动更新受 `maxSurge` / `maxUnavailable` 影响，稳定故障期常见 2 个旧 Running + 1 个新失败 Pod，共 3 个；等待或调整策略后数量可能再变。

步骤 2：用 describe 找出真正原因

```
$ kubectl describe pod web-7c9d4b6f8a-2k9xz | tail -8
Events:
  Type     Reason     Age                From     Message
  ----     ------     ----               ----     -------
  Normal   Pulling    30s (x4 over 90s)  kubelet  Pulling image "nginx:alpne"
  Warning  Failed     28s (x4 over 88s)  kubelet  Failed to pull image "nginx:alpne": ...
           rpc error: code = NotFound ... manifest unknown
  Warning  Failed     28s (x4 over 88s)  kubelet  Error: ErrImagePull
  Normal   BackOff    12s (x5 over 85s)  kubelet  Back-off pulling image "nginx:alpne"
```

解读：Events 里写得非常清楚：**`manifest unknown` = 仓库里没有这个标签的镜像。**这就是"证据"——排查不是猜，是找这一行。

四类常见拉镜像失败，靠 Events 区分：

- `manifest unknown` → 标签（版本）写错或不存在
- `not found` / `pull access denied` → 仓库名写错，或是私有镜像没配登录凭证
- `i/o timeout` / `dial tcp ... timeout` → 网络不通：**按 07 册的镜像加速器说明配一次**（注意：K8s 拉镜像不一定用 Docker——自建集群常用 containerd，加速要配 containerd 的 config.toml，而不是 Docker 的 daemon.json；沙箱环境按其说明）
- `403 Forbidden` → 走国内加速器时"标签不存在"常以 403 出现：同样先怀疑标签/名字写错（别只当"没权限"）

步骤 3：改回来，看它自己恢复

```
$ sed -i 's/nginx:alpne/nginx:alpine/' web.yaml
$ kubectl apply -f web.yaml
deployment.apps/web configured

$ kubectl get pods
NAME                   READY   STATUS    RESTARTS   AGE
web-6d8f7c9b4d-8vq4n   1/1     Running   0          15s
web-6d8f7c9b4d-q9m4t   1/1     Running   0          15m
```

解读：改回正确镜像名，**滚动更新**会自动把坏的那个换掉，全程不用你手动删 Pod。

步骤 4：把这次排查写成记录（这是产出）

```
# Pod 起不来排查记录
现象：kubectl get pods 显示 web-xxx  0/1  ImagePullBackOff
第一步：kubectl describe pod web-xxx | tail -20
        → Events 最后一行：Failed to pull image "nginx:alpne": manifest unknown
判读：不是集群问题，是镜像标签写错了（alpne ≠ alpine），仓库里没有这个标签
处理：把 deployment 的 image 改回 nginx:alpine → kubectl apply -f → 自动滚动更新
验证：kubectl get pods 两个副本都 Running；kubectl get deploy web 显示 2/2
一句话总结：ImagePullBackOff 先看 describe 的 Events，那里一定有一行写明原因
```

注意：最后那行报错的原文会随网络 / 镜像源不同——常见 `manifest unknown`，走加速器时也可能是 `502 Bad Gateway` 之类。认准 `Failed to pull image` 这一行就行，后面的字以你机器实际输出为准。

完成判定

· 能故意造出 ImagePullBackOff 并用 describe 找出原因 · 能说出四类拉镜像失败（标签错 / 权限或仓库错 / 网络不通 / 加速器下 403）各看什么关键词 · 能先预测再核对故障稳定期的新旧 Pod 状态，并解释为什么不是固定“只剩 1 个” · 能把它改回来并看到自动恢复 · 写出这份排查记录（放 ~/lab 或 learning-journey）

回滚方式：把镜像名改回正确值 `kubectl apply -f web.yaml`；沙箱关掉即清空。

6-06 迁移练习（先预测）：仍用 2 副本 Deployment，把镜像改成另一个不存在的标签，先写预测：应用后在哪条命令看到什么；为何故障稳定期可能同时有旧 Running Pod 和新失败 Pod；`describe` 里至少看哪一行才不是猜。执行后对照预测，再把镜像名改回并等待控制器恢复到 2/2。这里只交判断依据，不要求复述完整终端输出。

6-07 资源配额与调度：requests / limits

为什么这一关重要：前面几关练了 Pod，这一关补上**资源配额与调度**；它们决定多个工作负载共用一台节点时，谁会先被排上、谁会占得过多。

一、两个数字，两种作用（先分清）

字段	管什么	一句话
`requests`	**调度**：告诉 K8s"至少给我留这么多"	订座：没这个量的机器就不会把你排上去
`limits`	**上限**：超过就被限速（CPU）或杀掉（内存）	天花板：用到头就别再涨了

二、给 Pod 加上配额

```
$ cat > quota.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: hungry
spec:
  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "sleep 3600"]
      resources:
        requests:
          cpu: "2"            # 至少要 2 核
          memory: "4Gi"       # 至少要 4G 内存
        limits:
          cpu: "2"
          memory: "4Gi"
EOF
$ kubectl apply -f quota.yaml
pod/hungry created

$ kubectl get pod hungry
NAME     READY   STATUS    RESTARTS   AGE
hungry   0/1     Pending   0          15s        ← 起不来，挂着

$ kubectl describe pod hungry | tail -6
Events:
  Type     Reason            Age   From               Message
  ----     ------            ----  ----               -------
  Warning  FailedScheduling  20s   default-scheduler  0/2 nodes are available:
           2 Insufficient cpu, 2 Insufficient memory. preemption: 0/2 nodes are available
```

大机器上复现不了 Pending? 正常——这段输出是按 Killercoda 的小节点录的。你的机器核多、内存大时，把 requests 调大（如 `cpu: "100"` / `memory: "64Gi"`）再 apply，就会看到同样的 `Insufficient cpu`。

解读：**Pending = 调度器找不到能满足 requests 的机器。**Events 那句 `"Insufficient cpu"` 就是原因：沙箱里的节点没有 2 核空闲给它。**这就是"资源配额"在起作用 —— 它拦住了"抢不到座位还硬要坐"的 Pod。**

三、把请求改小，它就排得上了

```
$ sed -i 's/cpu: "2"/cpu: "100m"/; s/memory: "4Gi"/memory: "64Mi"/g' quota.yaml
$ kubectl delete pod hungry --ignore-not-found; kubectl apply -f quota.yaml   # --ignore-not-found：这个 Pod 不存在也不报错
pod/hungry created

$ kubectl get pod hungry
NAME     READY   STATUS    RESTARTS   AGE
hungry   1/1     Running   0          8s         ← 排上了
```

解读：`100m` = 0.1 核（m 是千分之一核）。**requests 写小 ≠ 只能用这么点**，它只是"订座"的门槛；真正的天花板是 limits。

四、不写 limits 会怎样（为什么生产上必须写）

```
# 一个没写 limits 的 Pod 内存泄漏 → 把整台节点的内存吃光
# 结果：同节点上别人的 Pod 被"饿死"或被驱逐（Evicted），故障从"一个服务"扩散成"一台机器"

$ kubectl describe node node01 | grep -A4 "Allocated resources"
  Resource           Requests    Limits
  cpu                250m (12%)  100m (5%)
  memory             128Mi (3%)   64Mi (1%)
```

解读：**不写 limits = 允许它随便吃。**生产上"一个人把节点撑爆、殃及邻居"是最典型的低级事故，所以团队规范一般要求：**每个容器都要有 requests 和 limits，内存 limit 尤其不能省。**

五、调度还看什么（先知道有这些）

```
kubectl get pod hungry -o wide          # NODE 那一列就是"调度到哪台"
kubectl describe node controlplane | head -20   # 看节点上的标签与状态
# 多节点时还能用 nodeSelector / affinity 把 Pod"指定"到某类机器上（GPT 机器、GPU 机器就靠这个区分）
```

解读：调度器先按 requests 过滤掉"装不下"的节点，再按标签/亲和性挑机器。**以后运维 GPU 机器时，"哪些 Pod 该上 GPU 节点"就是用这种办法控制的。**

完成判定

· 能说出 requests 和 limits 各自管什么（调度 vs 上限） · 能造出一个 Pending 并用 describe 说出"为什么排不上" · 能说出不写 limits 的后果（一个 Pod 拖垮整台节点） · 能用 `-o wide` 说出 Pod 被调度到哪台节点 · 能说出 `100m` 是什么意思

回滚方式：`kubectl delete pod hungry`（Pending 的 Pod 也删得掉）；沙箱关掉即清空。

6-08 阶段 6 • 学习总结练习：部署 → 弄坏 → 排查 → 留一份踩坑记录

业务场景

你在一个新集群上部署一个 Web 服务（2 副本）。上线后"看起来没问题"，但第二天同事说"服务好像是通的，但有时候访问不到"。领导要你：**不光修好，还要留下一份记录 —— 下次别人遇到同样的问题，照着你的记录就能解决。**

综合要求（这题把这册的技能串起来）

技能点：**Deployment / Service（6-02） + get/describe/logs 三只眼（6-03） + exec 进容器（6-04） + 自愈（6-05） + 拉镜像失败排查（6-06） + requests/limits 与调度（6-07）**

① 写 Deployment（2 副本） + Service，确认 `get all` 里都是正常的；

② **自己制造三个故障**（分别对应三类原因），每个都用三条眼查出根因：

- 镜像名写错（拉取类）
- requests 写得太大（调度类）
- 容器启动命令写错（应用类，比如 `command: ["sh", "-c", "/不存在的程序"]` 这种 **shell 包裹**的写法）

注意：直接写 `command: ["/不存在的程序"]` 会得到 `RunContainerError`（还没跑起来就失败），不是 `CrashLoopBackOff`——想看 CrashLoop 就用 shell 包裹，让容器"起来了又崩"。

③ 每修好一个，都要用 `kubectl get` 验证回正常；

④ 删掉一个 Pod 验证自愈，并截图/记录；

⑤ 写一份《Pod 起不来排查手册》（一页就够）：**状态名 → 大概率原因 → 先看哪条命令 → 常见关键词**；

（"Running 但访问不到"这一类的手册材料：`kubectl get endpoints 服务名`——若 ENDPOINTS 显示 <none>，说明 Service 的 selector 没匹配上任何 Pod：对照 `kubectl get svc 服务名 -o yaml` 里的 selector 与 Pod 的 labels 是否一字不差；再确认 targetPort 对准容器真正监听的端口——拿不准就用上面 ② 前那条临时 Pod 里 `wget 服务名` 的命令实测。）

⑥ 把这份记录放进 learning-journey（或 k8s-demo 仓库），README 里给出入口。

验收

· 三个故障都造出来过、都用 describe/logs 找到了根因 · 每个修复都有验证命令与输出 · 自愈实验有记录 · 有《Pod 起不来排查手册》，覆盖至少四种状态名（Pending / ImagePullBackOff / CrashLoopBackOff / Running 但访问不到） · 能不看笔记讲 3 分钟"我是怎么查的"

产出

仓库 `k8s-demo`（或 learning-journey 里的一个目录）：`web.yaml` + `README.md`（怎么跑） + `troubleshooting.md`（Pod 起不来排查手册） + 一两张 `kubectl get` 输出的截图。

AI 运维指向

在 AI 集群里，推理服务就是跑在这种 Pod 上：**拉不到镜像、资源不够排不上、启动就崩 —— 这三种故障占了日常的一大半。**你现在练的这套三条眼，就是以后排查推理服务的第一反应。

顺带一提 • 集群里还有这些东西（不用学，知道有这么回事就行）

是什么：ConfigMap / Secret（把配置和密码从镜像里拆出来）、Namespace（分区，6-01 见过）、Ingress（集群对外的统一入口，比 Service 更进一步）、StatefulSet（有状态服务，比如数据库）。

什么时候遇到：部署稍微正式一点的服务就会碰到 ConfigMap/Secret；"外网怎么访问我的服务"会碰到 Ingress。

真要用时去哪查：搜"kubectl create configmap 用法"、K8s 官方文档中文版；先知道名字，用到再看细节。

本册复习卡

① Deployment 保存期望副本，控制器持续把 `status` 拉回 `spec`；Service 通过 selector/labels 找到 Pod。

② 看到 Pending → 先查 `describe` 的 Events 调度原因；ImagePullBackOff → 查镜像名、标签或拉取证据；CrashLoopBackOff → 查容器日志和启动命令。

③ 独立变式：把副本数改为 3 并把镜像标签写错，先预测故障稳定时新旧 Pod 的可能数量，再列两条“旧服务尚在、新副本失败”的证据并执行；正文不提供答案。